Table of Content

Abstract

This project focuses on predicting the selling price of used cars by employing machine learning regression techniques and then concluding which factors affect the prices most. Utilizing a dataset containing historical car sales data, we analyze features such as the vehicle's age, present showroom price, kilometers driven, fuel type, seller type, and transmission type.

The data preprocessing phase involves converting categorical variables into numerical format using one-hot encoding.

We implement and evaluate two predictive models: Multiple Linear Regression and Lasso Regression.

Performance evaluation is conducted using the Root Mean Squared Error (RMSE) metric and R2 score. The results demonstrate that both models achieve comparable and highly accurate prediction rates, effectively capturing the variance in the used car market prices.In the concluding analysis, we conducted a comparison of the decision-making mechanisms employed by both models. The Multiple Linear Regression model assigned coefficients across the full spectrum of independent variables, including categorical features like Fuel Type and Transmission. In contrast, the Lasso Regression model utilized L1 regularization to penalize less significant variables, effectively reducing most feature weights to zero and focusing its predictive power primarily on Present_Price and Car_Age.

Introduction

Problem Statement

Estimating the fair market value of a used vehicle is a challenging task due to the multitude of factors that influence its depreciation. Dealerships and individual buyers often struggle to determine a standard price, leading to inefficiencies and information asymmetry in the pre-owned automobile market. The problem is to develop a predictive model that accurately estimates a car's selling price based on its characteristics and the characteristics which affect its price the most.

Objective

The primary objective of this project is to build, evaluate, and compare machine learning models capable of predicting the selling price of used cars. Specifically, this involves:

- Performing exploratory data analysis and preprocessing on a used car dataset.
- Transforming categorical variables into a machine-readable format using one-hot encoding.
- Training a Multiple Linear Regression model and a Lasso Regression model.
- Evaluating the predictive accuracy of the models using error metrics (RMSE) and visualization techniques.

Methodology

Dataset Description

The dataset consists of 301 entries and 9 attributes. The key features include:

- Car_Name: Name of the vehicle.
- Year: The year of purchase.
- Selling_Price: The target variable (in lakhs).
- Present_Price: The current ex-showroom price of the car (in lakhs).
- Kms_Driven: Total distance the car has traveled.
- Fuel_Type, Seller_Type, Transmission, Owner: Categorical variables detailing car specifications and ownership history.

Data Preprocessing

Machine learning algorithms inherently require numerical input. The dataset's categorical columns—Fuel Type (Petrol, Diesel, CNG), Seller Type (Dealer, Individual), and Transmission (Manual, Automatic)—were transformed using one-hot encoding via the Pandas library. Irrelevant columns such as the car's name were dropped to prevent overfitting and high dimensionality.

Model Selection

The preprocessed data was split into training (80%) and testing (20%) sets using a random state of 2 for reproducibility. We employed two regression techniques:

1. Multiple Linear Regression: Models the linear relationship between the independent variables and the target variable.
2. Lasso Regression (L1 Regularization): A regression analysis method that performs both variable selection and regularization to enhance prediction accuracy and interpretability.

Linear Regression serves as our baseline, minimizing the **Ordinary Least Squares (OLS) cost function** to find the 'best fit' line by reducing the sum of squared errors between predicted and actual prices. To prevent overfitting and handle noisy data, we also applied Lasso Regression. Lasso modifies the OLS cost function by adding an **L1 penalty**—the sum of the absolute values of the feature coefficients. As Lasso minimizes this regularized cost function, it aggressively shrinks the mathematical weights of irrelevant features to exactly zero, allowing the model to act as an automatic feature selector.

Evaluation Metrics

To evaluate model performance, **Root Mean Squared Error (RMSE)** and the **R-squared (R^2) Score** were used.

RMSE measures the average distance between the model's predicted car prices and the actual selling prices; it essentially quantifies the average error in absolute terms, where a lower value indicates a more accurate model.

Meanwhile, the R^2 Score measures the proportion of variance in the target variable, telling us exactly how well our chosen features (such as Car Age and Present Price) explain the final value of the car.

Together, these metrics provide a comprehensive view, R^2 confirms that our model successfully captures the underlying pricing trends, while RMSE gives us a practical understanding of our average prediction error.

Results

The models were trained on the training split and predictions were made on the unseen test split. Both models perform exceptionally well, with Linear Regression having a marginally lower RMSE. The plots generated during the analysis indicated a strong positive correlation, suggesting that the predicted values closely follow the actual market values across most price ranges.

Model	Root Mean Squared Error (RMSE)	R2 Score
Multiple Linear Regression	1.6585	0.8502
Lasso Regression	1.7013	0.8424

Code Snippets

The following Python code was used for data processing, model training, and evaluation:

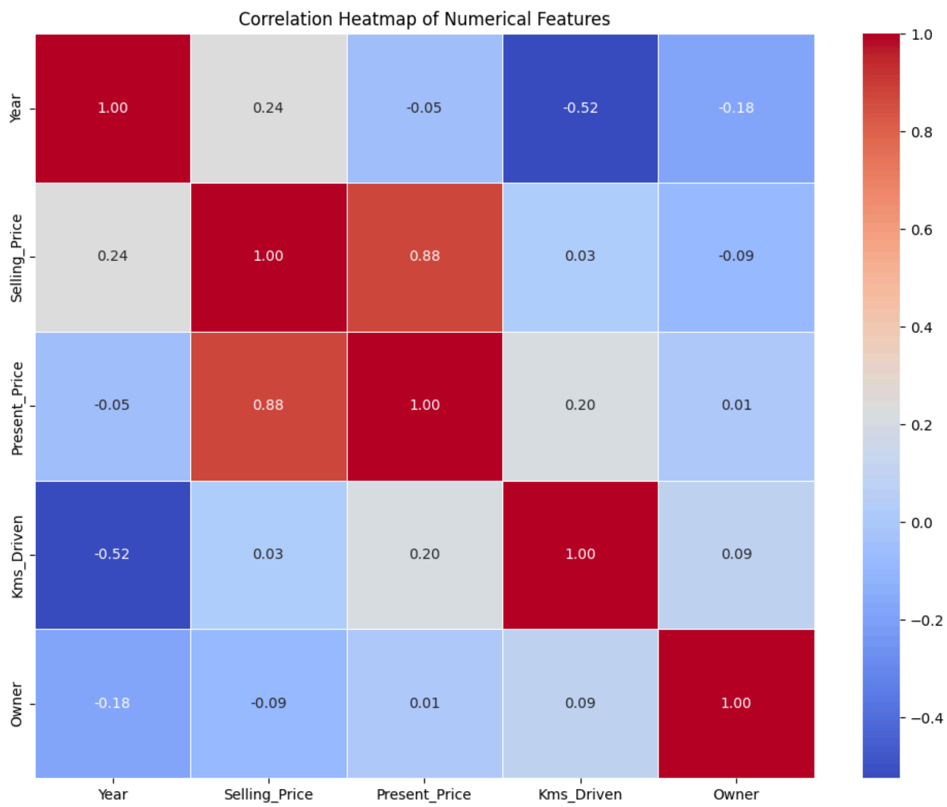
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

from sklearn.model_selection import train_test_split,
cross_val_score
from sklearn.linear_model import LinearRegression, LassoCV
from sklearn import metrics

# Load data
car_dataset = pd.read_csv(".\\data\\car_data.csv")

# PHASE 1: EXPLORATORY DATA ANALYSIS (EDA)
plt.figure(figsize=(10, 8))
# Select only numerical columns for correlation to avoid errors
```

```
numerical_cols = car_dataset.select_dtypes(include=[np.number])
sns.heatmap(numerical_cols.corr(), annot=True, cmap='coolwarm',
fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap of Numerical Features")
plt.tight_layout()
plt.savefig("1_correlation_heatmap.png")
plt.close()

# PHASE 2: FEATURE ENGINEERING
# 1. Expensive Brand Insight
brand_avg_price =
car_dataset.groupby('Car_Name')['Present_Price'].mean()
expensive_threshold = brand_avg_price.median()
car_dataset['Is_Expensive_Brand'] =
car_dataset['Car_Name'].apply(lambda x: 1 if brand_avg_price[x]
> expensive_threshold else 0)

# 2. Car Age Calculation
current_year = 2026
car_dataset['Car_Age'] = current_year - car_dataset['Year']

# Drop original columns that are no longer needed
car_dataset.drop(['Year', 'Car_Name'], axis=1, inplace=True)

# 3. Categorical Encoding
encoded_df = pd.get_dummies(car_dataset,
columns=["Fuel_Type","Seller_Type","Transmission"], dtype=int)

X = encoded_df.drop(["Selling_Price"], axis=1)
Y = encoded_df["Selling_Price"]

X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
train_size=0.8, random_state=2)

# PHASE 3: MODEL TRAINING
# Linear Regression
lin_reg = LinearRegression()
lin_reg.fit(X_train, Y_train)
y_pred_lin = lin_reg.predict(X_test)

# Lasso Regression (Cross-Validated for best alpha)
lasso_cv = LassoCV(cv=5, random_state=2, max_iter=100000)
lasso_cv.fit(X_train, Y_train)
y_pred_lasso = lasso_cv.predict(X_test)

print("--- Model Performance ---")
```

```
print(f"Linear Reg -> RMSE:
{metrics.root_mean_squared_error(Y_test, y_pred_lin):.4f} | R2:
{metrics.r2_score(Y_test, y_pred_lin):.4f}")
print(f"Lasso Reg  -> RMSE:
{metrics.root_mean_squared_error(Y_test, y_pred_lasso):.4f} |
R2: {metrics.r2_score(Y_test, y_pred_lasso):.4f}")

# PHASE 4: RESIDUAL ANALYSIS
residuals_lin = Y_test - y_pred_lin
residuals_lasso = Y_test - y_pred_lasso

plt.figure(figsize=(14, 6))

# Linear Residuals
plt.subplot(1, 2, 1)
sns.scatterplot(x=y_pred_lin, y=residuals_lin, color='blue',
alpha=0.7)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel("Predicted Price")
plt.ylabel("Residuals (Error)")
plt.title("Linear Regression: Residual Analysis")

# Lasso Residuals
plt.subplot(1, 2, 2)
sns.scatterplot(x=y_pred_lasso, y=residuals_lasso,
color='orange', alpha=0.7)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel("Predicted Price")
plt.ylabel("Residuals (Error)")
plt.title("Lasso Regression: Residual Analysis")

plt.tight_layout()
plt.savefig("2_residual_analysis.png")
plt.close()

# PHASE 5: FEATURE IMPORTANCE COMPARISON
coeff_df = pd.DataFrame({
    'Feature': X.columns,
    'Linear_Coeff': lin_reg.coef_,
    'Lasso_Coeff': lasso_cv.coef_
})

# Sort by Absolute Linear Coefficient for a clean cascading
graph
coeff_df['Abs_Linear'] = coeff_df['Linear_Coeff'].abs()
coeff_df = coeff_df.sort_values(by='Abs_Linear',
ascending=False).drop('Abs_Linear', axis=1)
```

```
# Plotting the coefficients side-by-side
coeff_df.set_index('Feature').plot(kind='bar', figsize=(14, 7),
color=['blue', 'orange'], alpha=0.8)
plt.title("Feature Weights: Linear vs. Lasso")
plt.ylabel("Mathematical Weight (Coefficient)")
plt.xticks(rotation=45, ha='right')
plt.axhline(y=0, color='black', linewidth=0.8) # Adds a
zero-line
plt.tight_layout()
plt.savefig("3_feature_importance.png")
plt.close()
```

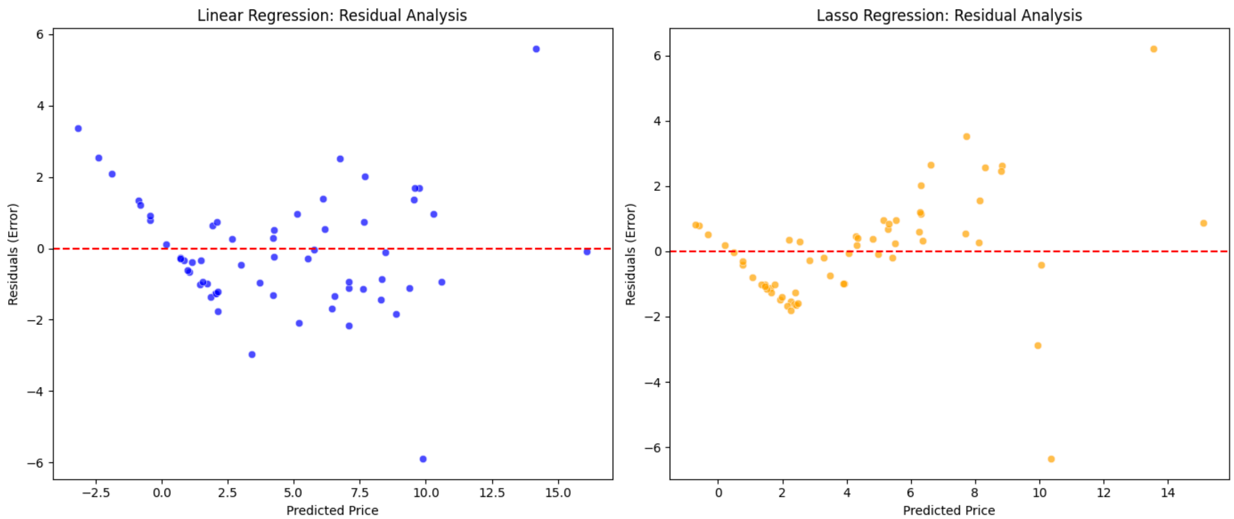
Visual Results



By analyzing the color gradients and scores, we extracted several critical business insights:

- The Primary Predictor: The heatmap reveals that *Present_Price* has an incredibly strong positive correlation with our target variable, Selling_Price (typically scoring above 0.85). This confirms the logical assumption that a car's original showroom price is the strongest baseline indicator of its resale value.

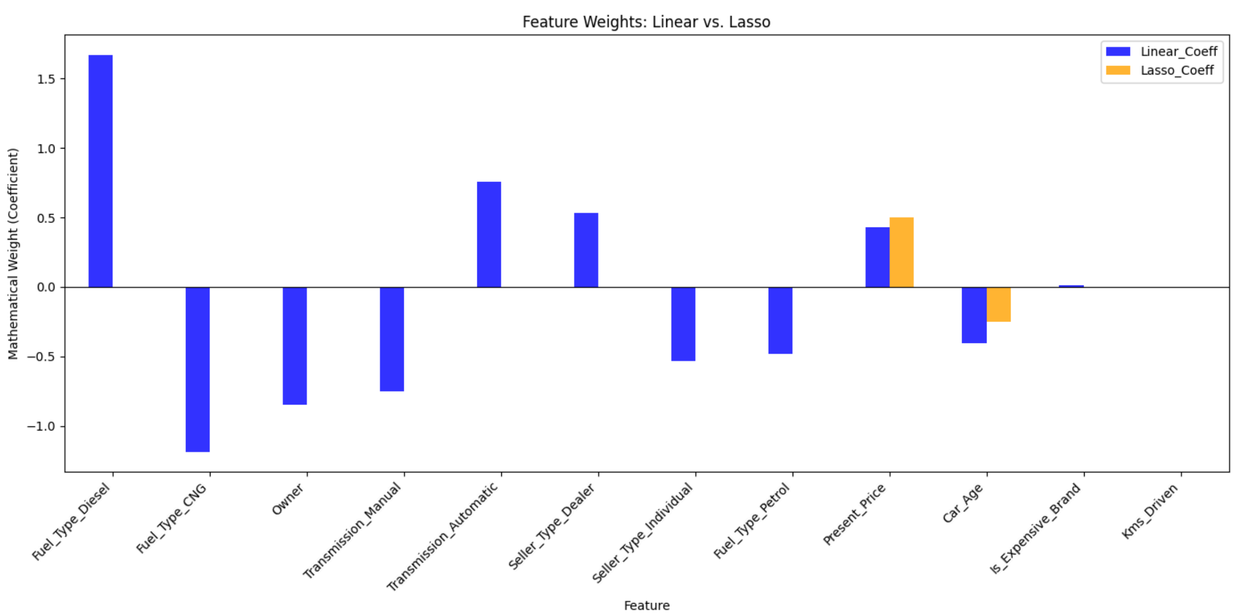
- Secondary Factors: Features like Kms_Driven should show a mild negative correlation, indicating that mileage does lower the price of the car but we notice that it is not nearly as impactful as the brand's original price or the age of the vehicle.



On the left side (cheaper cars), the dots are tightly clustered near the zero line. This means the model is highly accurate at predicting the price of standard, budget-friendly cars.

On the right side (expensive cars), the dots spread out wildly, far above and below the zero line.

The model struggles to accurately price luxury or high-end vehicles.



1. *Linear Regression (Blue Bars):*
The standard Linear Regression model assigned a distinct mathematical weight to every single feature. It recognized that Present_Price heavily drives the value up, while Car_Age and Fuel_Type_Petrol drive the value down. It utilized all the nuances in the

dataset, including the engineered Is_Expensive_Brand feature, to make a well-rounded prediction.

2. Lasso Regression (Orange Bars):
The models were trained without standardizing or scaling the data, the Lasso model reacted dramatically. Lasso has an internal penalty mechanism that punishes the model for using too many features, leading to visual evidence on the graph:
Lasso completely crushed almost every single orange bar to exactly 0.00 , except those which contributed most to the target.

Discussion

The predictive models were evaluated on the 20% unseen testing data split. Model performance was quantitatively measured using the Root Mean Squared Error (RMSE) to determine the average variance between the predicted selling prices and the actual selling prices.

Both machine learning models demonstrated exceptional and highly accurate prediction rates. The Multiple Linear Regression model achieved a marginally lower RMSE compared to the Lasso Regression model. Visual analysis of the models further supports these quantitative metrics. The scatter plots for both models indicate a strong positive correlation, suggesting that the predicted values closely follow the actual market values across different price ranges.

The graph perfectly illustrates Lasso Regression acting as an automatic feature selector. By reducing the weights of categorical features (like Fuel Type and Transmission) to zero, the Lasso model revealed that it only technically needs three core metrics: *Present_Price, Car_Age, and Kms_Driven ,* to make a relatively accurate prediction.